

Bidirectional Translation Between Natural Language and SPARQL

Engin Samet Dede

Sevgi Dilay Demirci

Kamar El Morabit

Jumainah Khan

Saïfudeen Hisham Kolikara Mahin

Group B (Lamia)

University of Geneva

Course: Knowledge Organization Systems

Abstract

This project presents a bidirectional Natural Language and SPARQL translation system designed to make large public knowledge graphs easier to access. The system allows users to ask questions in ordinary English and automatically generates SPARQL queries for Wikidata or DBpedia. It also supports the reverse direction by explaining existing SPARQL queries in natural language, helping users understand what a query is doing before or after execution.

The prototype is built around a Streamlit interface, domain specific YAML configuration files, and large language models accessed through Hactar and Hugging Face. The main design choice is to guide the models with endpoint-specific prefixes, ontology hints, and few-shot examples rather than fine tuning them. This makes the system easier to extend to new domains while keeping the query generation process transparent.

The project also includes query cleaning, retry mechanisms, result summarisation, knowledge graph visualisation, and an evaluation framework for comparing models across multiple domains. The evaluation shows that both Qwen and Llama can often generate executable and semantically meaningful SPARQL queries, but failures still occur because of hallucinated identifiers, endpoint timeouts, schema mismatches, and noisy knowledge graph data. Overall, the project demonstrates that LLM based SPARQL generation is useful for lowering the access barrier to knowledge graphs, but it still requires careful domain configuration, deterministic post-processing, and transparent evaluation.

Contents

1	Introduction	4
1.1	Motivation and Problem Statement	4
2	Background	4
2.1	Knowledge Organization Systems	4
2.2	The Semantic Web Stack and Linked Data	5
2.3	SPARQL	5
2.4	Wikidata and DBpedia	6
2.5	Large Language Models	6
3	System Design	7
3.1	Architecture Overview	7
3.2	User Interface	9
3.3	Domain Configuration Files	13
3.4	Model Arena	14
4	Methodology	16
4.1	Domain Routing	16
4.2	Entity and Predicate Resolution	19
4.3	NL $\rightarrow$ SPARQL	20
4.4	SPARQL $\rightarrow$ NL	22
4.5	Prompt Engineering Strategies	24
4.6	Knowledge Graph	25
5	Implementation	29
5.1	Technology Stack	29
5.2	Hactar LLM Service	30
5.3	HuggingFace	30
5.4	Handling Endpoint Quirks	30
6	Evaluation	32
6.1	Evaluation Setup	32
6.2	Question Set	33
6.3	Quantitative Results	33
6.4	Qualitative Analysis	33
6.5	LLM as a Judge	35
7	Discussion	36
7.1	Limitations	36
7.2	Future Work	37
8	Conclusion	37

1 Introduction

1.1 Motivation and Problem Statement

Large public knowledge graphs such as Wikidata and DBpedia contain a huge amount of structured information on many variety of domains. In theory, this information is openly accessible. In practice, however, accessing it is not simple for most users, because the information has to be retrieved through SPARQL queries.

Writing SPARQL requires more than just knowing what question to ask. The user also needs to know the syntax of the query language, the correct endpoint, the relevant prefixes, and the specific classes and properties used by each knowledge graph. For a non-technical user, this creates a clear barrier: the knowledge is available, but it is difficult to reach.

This project addresses that problem by building a system that translates between natural language and SPARQL. In the natural language to SPARQL direction, the user asks a question in ordinary English, and the system generates a query for the selected knowledge graph. In the SPARQL to natural language direction, the system explains an existing query in a more readable form, helping the user understand what the query is doing. Since it is possible that a small mistake can lead to an invalid query, an empty result, or a correct-looking query that answers the wrong question, much of the project focuses on domain specific prompting, query cleaning, execution, and evaluation.

Contributions. The main contributions of this project are: (i) a working bidirectional NL $\leftrightarrow$ SPARQL prototype covering ten domains across Wikidata and DBpedia; (ii) a two-phase entity and predicate resolution step that replaces static, hand-curated identifiers with API-backed lookups at query time; (iii) a per-domain YAML configuration system that extends to new domains without code changes; (iv) a multi-model arena with structured user feedback and a composite evaluation framework that combines structural metrics with an LLM-as-a-Judge layer.

Paper organisation. Section 2 covers the technical background on knowledge graphs, SPARQL, and language models. Section 3 describes the overall architecture and user interface. Section 4 details the core design choices. Section 5 covers the technology stack and practical engineering issues encountered. Section 6 presents the evaluation setup, results, and qualitative analysis. Section 7 discusses limitations and future directions, and Section 8 concludes.

2 Background

2.1 Knowledge Organization Systems

Knowledge organization systems (KOS) range from simple term lists and thesauri at one end to formal ontologies at the other, with the key difference being how explicitly semantic relationships are defined [5, 21]. Wikidata and DBpedia sit at the formal end of that spectrum. Both use globally unique URIs for entities, define properties with domains and ranges, and organise data into class hierarchies that can be queried formally [9, 17].

The practical consequence is that these knowledge bases are machine-readable but not directly accessible to users without training. Retrieving information requires knowing SPARQL, the local property naming conventions, and enough of the data model to write structurally correct queries. Making that access easier is the core motivation for this project. The two graphs also differ from each other in ways that matter technically: Wikidata identifies everything with opaque Q-IDs and P-IDs, while DBpedia uses readable names like `dbo:director`, so any system that supports both has to manage two different vocabularies and two different endpoint behaviours.

2.2 The Semantic Web Stack and Linked Data

The Semantic Web is built on a layered set of standards. RDF (Resource Description Framework) is the base layer: it represents any statement as a subject-predicate-object triple, where each element is either a URI or a literal. Because URIs are globally unique, RDF datasets from different sources can be combined without any schema-matching step. RDFS and OWL build on top of RDF to add class hierarchies, property constraints, and logical axioms, making it possible to define and reason over formal ontologies.

Bizer, Heath, and Berners-Lee formalised how to publish and interconnect RDF data through four Linked Data principles: name things with URIs, make those URIs look-upable over HTTP, return RDF when they are looked up, and include links to related URIs [2]. Wikidata and DBpedia are among the most central nodes in the Linked Open Data cloud that these principles created. Both expose public SPARQL endpoints, which is where this project connects to the stack. SPARQL queries a graph by describing a pattern of triples to match against the stored data [18]. It is powerful enough to express complex questions, but it requires the user to know the structure of the target graph in detail. That gap between what a user wants to ask and what SPARQL needs to execute is exactly what this project addresses.

2.3 SPARQL

SPARQL is the standard query language for RDF graphs, standardised by the W3C [18]. Where SQL queries tables, SPARQL queries a graph of triples by describing a pattern of triples to match. A query is built from a basic graph pattern, a set of triple patterns in which any of the subject, predicate, or object positions can be a variable. The query engine finds every subgraph of the data that matches the pattern and binds the variables accordingly.

A query uses one of four forms. **SELECT** returns a table of variable bindings, **ASK** returns a boolean, **CONSTRUCT** builds a new RDF graph from the matches, and **DESCRIBE** returns RDF about a resource. On top of basic graph patterns, SPARQL adds **FILTER** for value constraints, **OPTIONAL** for matches that may or may not be present, **UNION** for alternatives, and aggregation with **GROUP BY**. These constructs are what make SPARQL expressive enough to ask real questions, but they also make it hard to generate correctly, Pérez et al. provide a compositional semantics for the core of SPARQL and show that the evaluation problem for general SPARQL graph patterns is PSPACE-complete, with this high complexity arising from the unrestricted use of nested **OPTIONAL** patterns [12]. This is the formal reason behind the feasibility split in the project brief queries with clear subject predicate object structure are easy to produce, while nested **OPTIONAL** and

UNION queries are genuinely hard, for machines and language models alike.

For this project SPARQL is the target language, every natural-language question the user asks has to be turned into a syntactically valid and semantically correct SPARQL query before it can be executed against a knowledge graph.

2.4 Wikidata and DBpedia

Wikidata and DBpedia are both large, general-purpose knowledge graphs, but they are built in fundamentally different ways, and our system has to work with both.

Wikidata is a collaboratively edited knowledge base where every entity is an *item* with an opaque identifier, and every relationship is a *property* with an identifier. Data is modelled as statements that can themselves carry qualifiers and references, which makes the model expressive but also means that the human readable name of an entity is not part of its identifier; labels are stored separately and are multilingual. In practice this is why Wikidata queries rely on the dedicated SERVICE `wikibase:label` service to attach readable names efficiently.

DBpedia takes the opposite approach. Instead of being edited directly, it is extracted automatically from Wikipedia infoboxes and articles [1, 9]. Entities are identified by readable URIs under the `dbr:` namespace (for example `dbr:The_Matrix`) and the ontology vocabulary lives under `dbo:`. Because it is derived from English Wikipedia, DBpedia is English-centric and its coverage and consistency depend on how well the underlying infoboxes were filled in, which is the source of the data-quality quirks we describe in Section 5.4.

The consequence is that the same conceptual question produces a completely different SPARQL query depending on the endpoint: different identifier schemes, different prefixes, different property names, and different performance characteristics. A system that wants to answer questions across several domains cannot commit to just one of them, because some domains are far better represented in Wikidata and others in DBpedia. This is exactly why our domain configuration (Section 4.1) binds each domain to a specific endpoint and to the vocabulary that endpoint expects.

2.5 Large Language Models

Large language models (LLMs) are neural networks trained on very large text corpora to predict the next token. The property that makes them useful here is *in-context learning*: a sufficiently large model can perform a new task purely from instructions and a few examples in the prompt, without any change to its weights. Brown et al. demonstrated this with GPT-3, showing that few-shot prompting could reach competitive performance on tasks the model was never explicitly trained for [3]. This is the foundation our system relies on: we never fine-tune a model: we shape its behaviour entirely through prompt construction and few-shot examples.

Instruction-tuned models extend this by being further trained to follow natural-language instructions, which makes them far more controllable for a task like “translate this question into SPARQL”. The LLaMA family [15, 16] is particularly relevant to this project because the weights

are openly available, which is what makes it possible to host the models locally rather than calling a commercial API. A second useful technique is chain-of-thought prompting, where the model is asked to reason step by step before producing an answer; Wei et al. showed this substantially improves performance on tasks that require intermediate reasoning [19], which is directly applicable to building a multi-clause SPARQL query from an ambiguous question.

In this project the models are served through Hactar, a locally hosted LLM service run by the University of Geneva. Running models locally has two advantages over a commercial API that matter for a knowledge-organisation setting: prompts and any sensitive data stay inside the university infrastructure, and the served model versions are stable, which keeps prompt-engineering experiments reproducible over the lifetime of the project. The trade-off is throughput and availability, which is the practical reason we later added a HuggingFace fallback backend for the evaluation (Section 6).

3 System Design

3.1 Architecture Overview

This application’s system is structured as a linear pipeline. The main steps are: First a request is sent to the Streamlit front-end [14], then it is processed step by step until finally it arrives at one of two knowledge graph endpoints — Wikidata [17] or DBpedia [9] — from where it is transferred to the front-end and presented as a human-readable argument. Each of these steps is a unique Python module so that each one can be changed or expanded without impacting other steps.

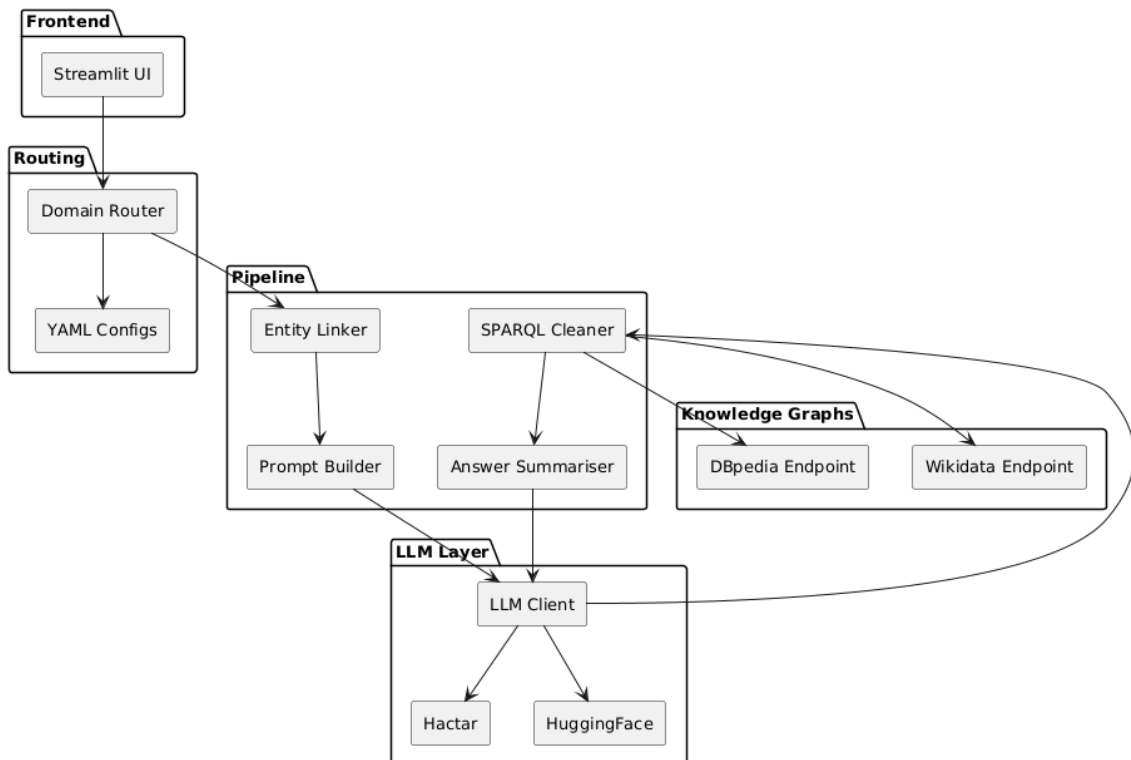


Figure 1: High-level component architecture of the NL↔SPARQL prototype.

The entry point is the Streamlit UI, where the user types their question and selects their domain and then passes on to the domain router. The router loads the corresponding domain configuration, a YAML file that includes the endpoint URL, prefix for the namespaces, ontology hints, and few-shot question–SPARQL pairs. This file is the extension point of the system: no code changes are required for adding a new domain, just a new YAML file.

After resolving the domain, the pipeline enters the entity and predicate resolution step. A lightweight LLM call first extracts the named entities and relational keywords from the user’s question; those terms are then sent to the Wikidata search API, which returns verified QIDs and PIDs (Section 4.2). This replaced an earlier design that relied on QIDs and PIDs hardcoded in the YAML domain configuration files. When both entity and predicate identifiers are resolved, the pipeline attempts a deterministic template query directly against the SPARQL endpoint [18], skipping the main LLM call entirely. If that template query returns results, the pipeline returns immediately. Otherwise, the verified identifiers are passed to the prompt builder, which assembles the full prompt from system instructions, domain hints, few-shot examples, and the grounded entity block. The prompt is then sent to the LLM client, which routes it to Hactar or HuggingFace [15] depending on availability and the user’s model selection.

Finally, the model output is cleaned removing any markdown or prose and is then executed against the domain’s SPARQL endpoint. In case the query executes successfully but yields no results, the pipeline retries once by calling the LLM again with the same prompt but at a higher temperature (0.7), allowing the model to produce a different query. A successful result is handed to the answer summariser, which generates a concise, natural language answer and the raw result table.

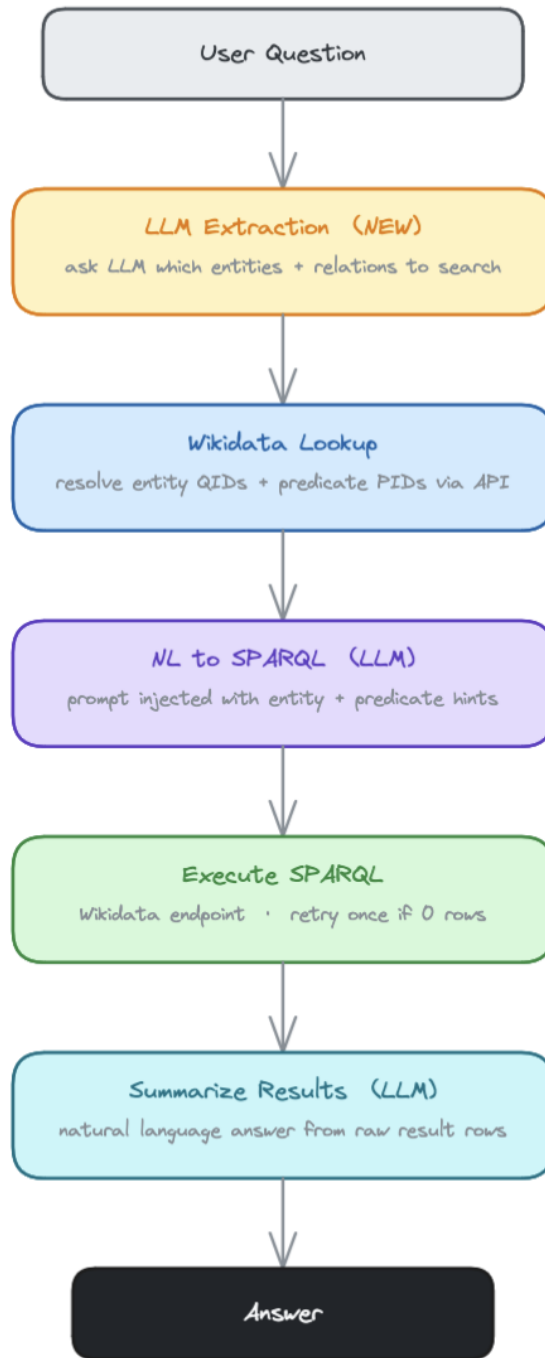


Figure 2: Updated pipeline activity diagram showing the new LLM extraction and Wikidata lookup steps, execution flow, and retry logic.

3.2 User Interface

The interface is a single-page Streamlit application. The left side of the screen is a fixed sidebar with the controls that apply to the whole session, and the rest of the screen is a set of tabs, one for each task. There are five tabs in total. This section covers the three that do the actual

translation and visualisation work.

The sidebar is the same on every tab. At the top is the *Language Model* dropdown, which decides which LLM runs every translation. Right below it is a status panel that lists the Hactar models and the HuggingFace models, each with an **ONLINE** or **OFFLINE** label and a refresh button. We added this panel after getting tired of the same problem ourselves: Hactar goes down fairly often (Section 5.2), and if the status is not visible you only find out a model is dead after you have already typed a question and waited for it to fail. Seeing which models are up before running saves that wasted round-trip. Under *Options* there is one toggle, *Summarize results in plain English*, which decides whether the user gets just the raw result table or also a natural-language answer on top of it. We made this a choice rather than always-on because the extra summary costs another LLM call and not everyone wants it. Below that are a dark-mode toggle and the two endpoint URLs, with the active endpoint shown in bold and the inactive one greyed out so it is always clear where a query is going.

Natural Language to SPARQL. This is the main tab and the one most users will spend their time in. Above the input box there is a grid of example cards. Each card has a domain tag (SCIENTISTS & NOBEL PRIZES, MOVIES, MUSIC, and so on), a difficulty tag (SIMPLE, LOOKUP, FILTER, JOIN, AGGREGATE, . . .), the example question, and a *Use this* button that drops the question straight into the input. The reason these cards are there is simple. An empty text box tells the user nothing about what the system can do or how to ask for it, and most people will not guess the phrasing that works best. The cards show the range of each domain and a question shape that is known to work. The difficulty tags are also a quiet warning: a SIMPLE list almost always works, while an AGGREGATE or JOIN question is harder and fails more often. Underneath the question box is the domain dropdown. The figure below shows the tab with the sidebar on the left and the example grid filled in.

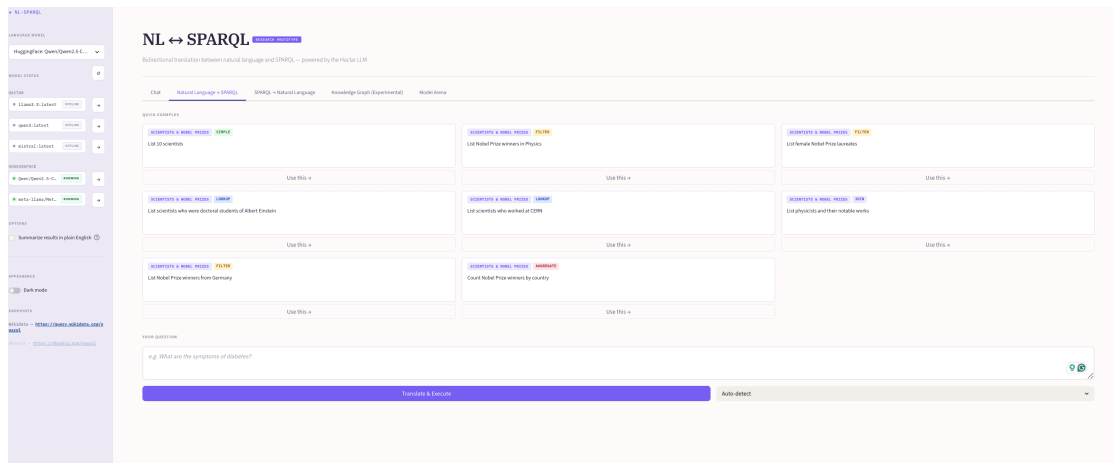


Figure 3: The Natural Language → SPARQL tab, showing the global sidebar and the per-domain example cards.

The domain dropdown is worth a closer look because it carries a design decision from Section 4.1. The options are not a flat list. They are split into a *Wikidata* group and a *DBpedia* group with a

separator line between them.

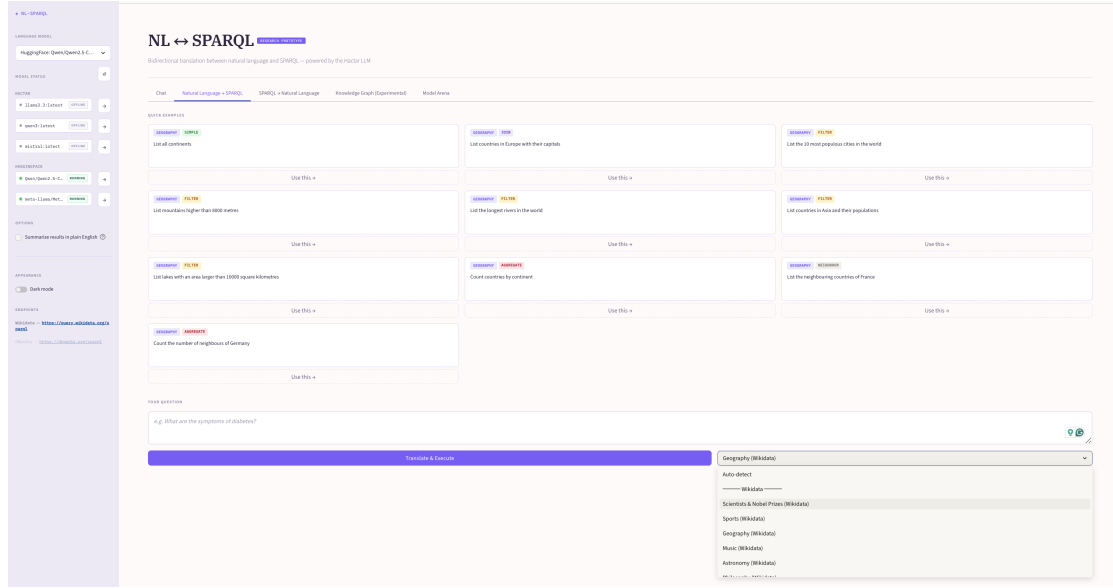


Figure 4: The domain selector, grouped by knowledge base so the Wikidata/DBpedia split is visible at the point of selection.

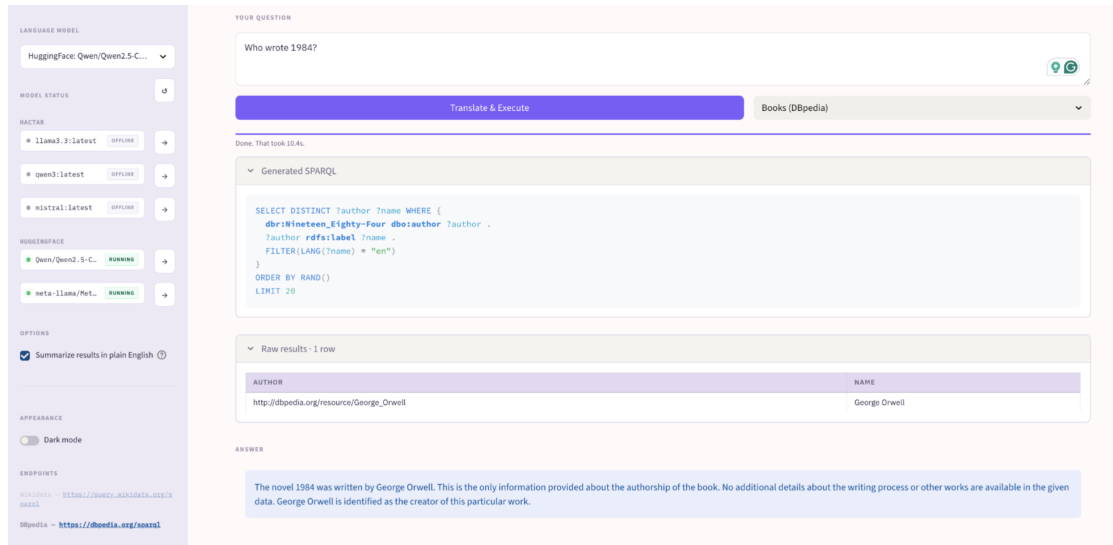


Figure 5: Example execution of the Natural Language → SPARQL pipeline in the Books domain using DBpedia. The system answers the question “Who wrote 1984?” by generating and executing a SPARQL query that retrieves George Orwell as the author.

SPARQL to Natural Language. This tab is the first one turned around. It has the same kind of example grid, except each card now has a *Load query* button that fills the box with a ready-made SPARQL query, one per domain. The thinking is the same as on the other tab, just pointed the other way: someone who wants to try the verbaliser probably does not have a SPARQL query lying around to paste in, so we give them one for every domain. Below the grid is the query box and an *Explain in English* button that runs the hybrid verbaliser from Section 4.4.

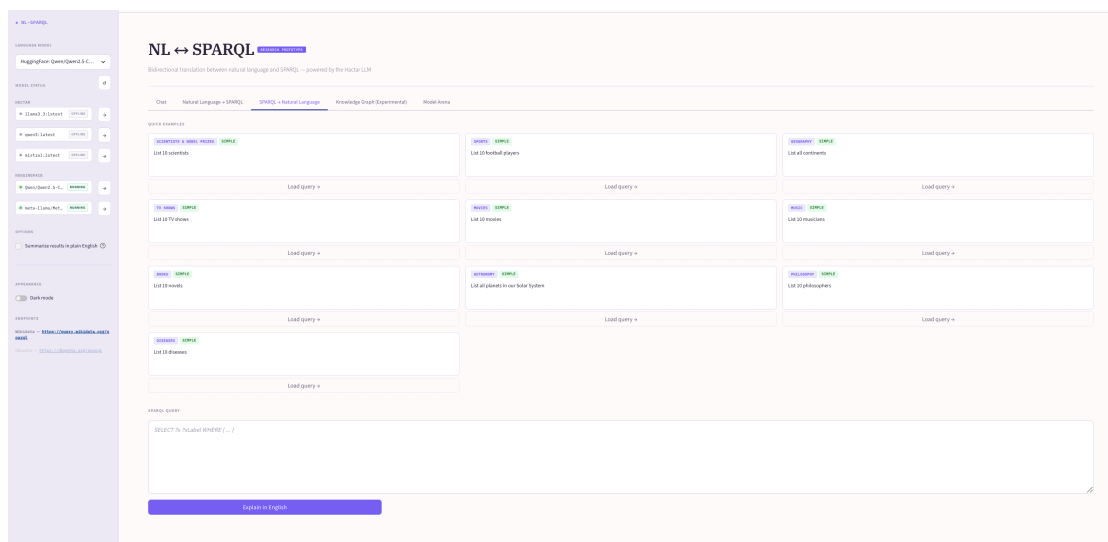


Figure 6: The SPARQL $\rightarrow$ Natural Language tab, with one ready-made query example per domain.

Knowledge Graph (Experimental). The last tab draws the results of a query as an interactive network. Before the user runs anything there is a short note explaining the two modes, because the output shape depends on the query and we would rather say that than have the user be surprised by it. A query with two or more entity columns becomes a relationship graph, and a flat single-column list becomes a star graph around a central concept node (the reasoning behind the two modes is in Section 4.6). A row of example buttons fills the query box with something that works, and after pressing *Visualize* the graph appears with a one-line summary above it — node count, edge count, average degree, and the most connected node, for example *Aristotle* in the philosopher-influence example. That summary line is there so the user gets a quick sense of the graph without having to read it node by node. A search box helps find a node in a crowded graph, a *Download Turtle* button exports the same data for Protégé (Section 4.6), and the raw table and a path-finder are tucked into collapsible panels below so they do not get in the way.

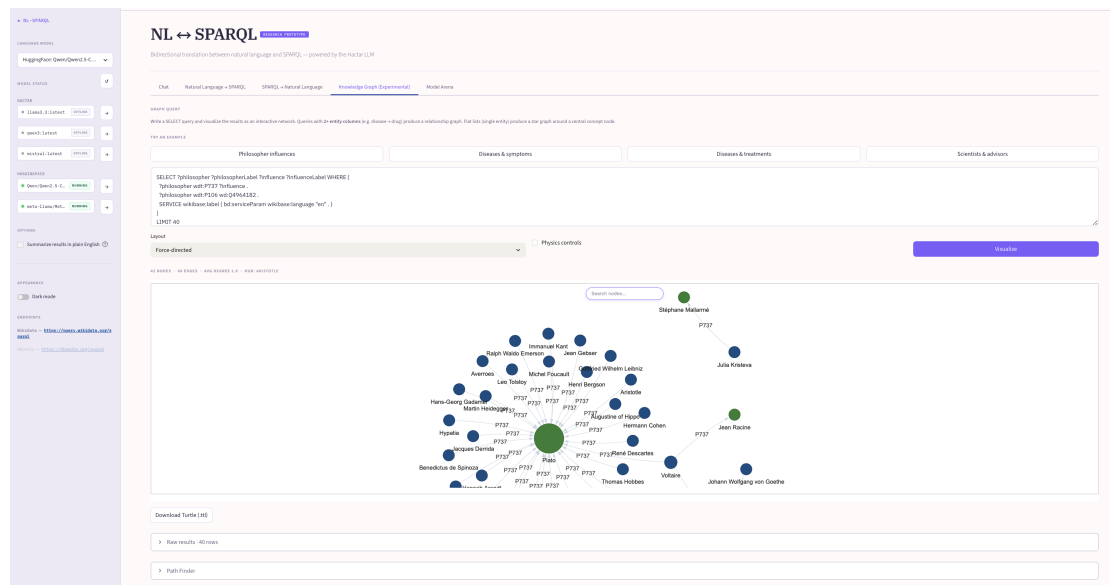


Figure 7: The Knowledge Graph tab rendering the philosopher-influence query as a star graph, with the summary line and Turtle export.

3.3 Domain Configuration Files

There is one YAML file in the `configs/` directory for each domain in the system. The file contains all the elements that a pipeline should know to deal with that domain: the URL of the SPARQL endpoint [18], the set of namespace prefixes to be used in the automatically generated queries, a block of ontology hints that provides a description of the most relevant classes and properties, a list of few-shot question–SPARQL pairs [3] that are presented as an example in the prompt, and a list of example questions that are displayed in the UI. To add a new domain, it is enough to drop in a new YAML file; no code changes are required. The split between the domain and pipeline code is deliberate: the prompt engineering all resides in the YAML file, allowing a domain expert to tweak or expand a domain without touching any Python.

```
domain_name: "Books (DBpedia)"
description: "Literature domain using DBpedia: ..."
endpoint: "https://dbpedia.org/sparql"

prefixes: |
  PREFIX dbo: <http://dbpedia.org/ontology/>
  PREFIX dbr: <http://dbpedia.org/resource/>
  PREFIX dbp: <http://dbpedia.org/property/>

ontology_hints: |
  Key DBpedia classes and properties:
  - dbo:Book = "a book"
  - dbo:Writer = "a writer or author"
  - dbo:author = "author of the book"
  - dbo:genre = "literary genre"

few_shot_examples:
  - question: "Who wrote 1984?"
```

```

tag: "Lookup"
sparql: |
    SELECT DISTINCT ?author ?name WHERE { ... }

example_questions:
- question: "List 10 novels"
  tag: "Simple"

```

Listing 1: Skeleton of a domain YAML configuration file.

Currently, the system has 10 domains in the two knowledge graphs. Three target DBpedia:

- **Books:** which focuses on authors, genres and publishers with the `dbo:Writer` and `dbo:Book` classes
- **Movies:** which focuses on directors, actors, release dates and box office data
- **TV Shows:** which focuses on series, seasons, episodes, cast and broadcast networks

The other seven are:

- **Diseases:** condition, symptom, treatment and ICD-10 codes from the biomedical domain
- **Music:** artists, bands, albums, songs, and genres
- **Philosophy:** philosophers, philosophy schools, and influences
- **Sports:** athletes, teams, competitions
- **Scientists & Nobel Prizes:** researchers, laureates, institutions
- **Astronomy:** planets, stars, moons, galaxies, and other celestial bodies
- **Geography:** countries, cities, continents

The DBpedia domains [1] have prefix `dbo:`, `dbp:`, and `dbp:` and depend on the readable names of properties, whereas the Wikidata domains [17] use entity-looked-up Q-IDs, with P-IDs.

3.4 Model Arena

The Model Arena tab allows the user to run the same natural-language question through several language models simultaneously and compare their outputs side by side. The motivation is straightforward: no single model is reliably best for all question types, and the only honest way to assess this is to run them in parallel on the same input and inspect the results together.

Model selection and parallel execution. At the top of the tab the user selects which models to include using a multiselect widget. This step is practical rather than aesthetic: some servers are intermittently unavailable, and excluding a known-down model prevents the whole comparison from stalling. Once the selection is confirmed and the question is submitted, each model runs

in a separate thread using Python’s **ThreadPoolExecutor**, so the slowest model does not block the others. Each thread follows the same pipeline as the main translation tab: the question is translated to SPARQL, the query is cleaned and executed, and if the endpoint returns an error or zero rows the system automatically retries using a label-based fallback prompt. The per-model wall-clock time is recorded for comparison.

Per-model result panel. After execution, each model is shown in its own expandable panel (Figure 8). The panel displays the generated SPARQL query, a one-sentence explanation of what the query retrieves (produced by a second LLM call), the result rows returned by the endpoint, and a natural-language answer synthesised from those rows. A *Retried* badge appears when the fallback prompt was needed.

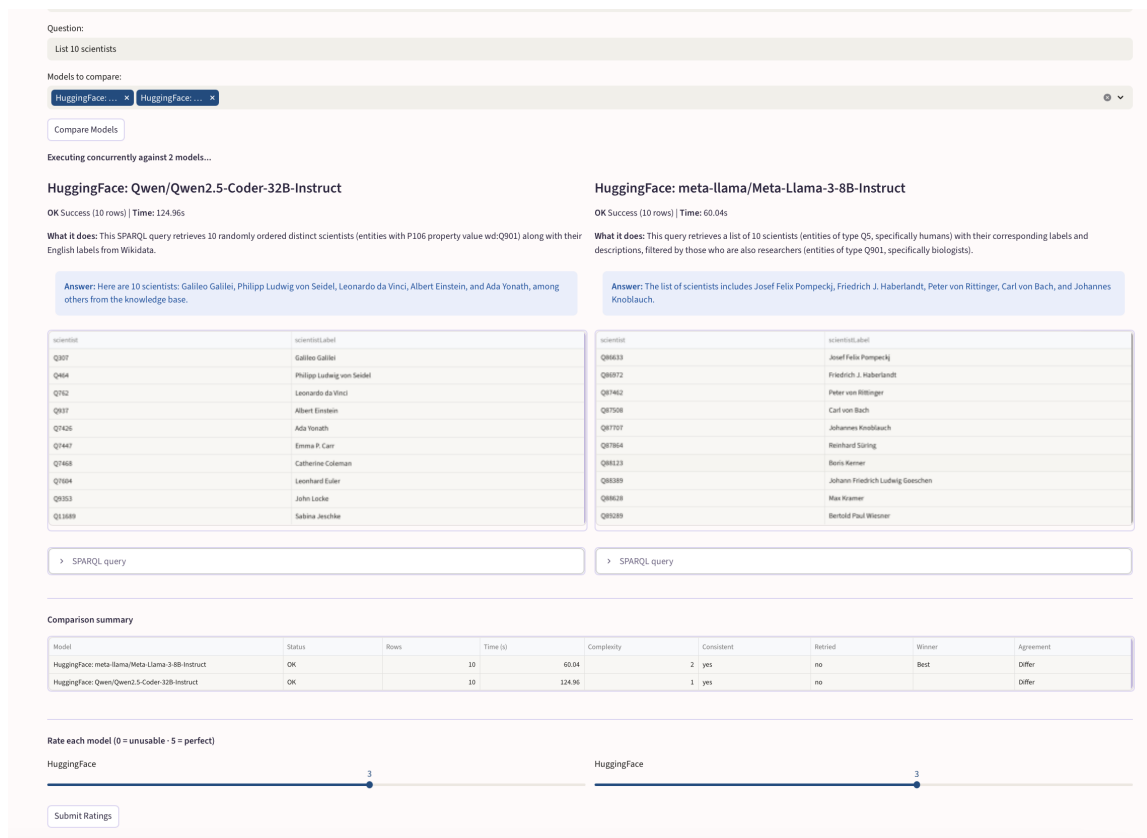


Figure 8: The Model Arena tab after running a question through two models. Each model panel shows its SPARQL query, a one-sentence explanation, the result rows, and a natural-language answer. The rating sliders appear below.

Comparison summary table. Below the individual panels, a summary table brings all models together in one view. It reports, for each model: execution status, number of rows returned, wall-clock time, SPARQL complexity (number of triple patterns), whether the answer was consistent across two independent runs of the same question, whether a retry was needed, a *Best* badge for the fastest successful model, and whether all models agreed on the same first answer.

User ratings and feedback. After reviewing the results, the user can rate each model on a 0–5 scale (Figure 9). On submitting the ratings, a second step collects structured feedback: for

models rated 3 or higher, a set of positive reasons is shown (e.g. *Correct answer*, *Clear explanation*, *Good SPARQL structure*); for models rated below 3, a set of negative reasons is shown (e.g. *Wrong or empty answer*, *Bad SPARQL*, *Too slow*). The user can also type a free-text comment for each model (Figure 10). All ratings and feedback are appended to a local JSON file so that the leaderboard, shown at the bottom of the tab, persists across sessions.

Tell us more about your ratings

HuggingFace — 4/5

What did this model do well?

Choose options

- Select all
- Correct answer
- Clear explanation
- Useful results
- Good SPARQL structure
- Fast response

Submit Feedback

Figure 9: The feedback step for a model rated 4 out of 5. The user selects reasons from a predefined list and can add a free-text comment.

Tell us more about your ratings

HuggingFace — 4/5

What did this model do well?

Choose options

Submit Feedback

HuggingFace — 4/5

What did this model do well?

Choose options

Submit Feedback

Figure 10: The feedback form showing both models side by side, each with its rating, reason checkboxes, and comment field.

4 Methodology

This section describes the methodology used to build the bidirectional Natural Language to SPARQL system, covering the three main components: domain routing (which lets the user choose the domain from a dropdown grouped by knowledge base), Natural Language to SPARQL translation, and SPARQL to Natural Language translation.

4.1 Domain Routing

Domain routing is the first step of the pipeline because the selected domain determines how the rest of the system behaves. Before a SPARQL query can be generated, the system needs to know which knowledge base, endpoint, prefixes, ontology hints, and examples should be used. For this reason, domain routing acts as a bridge between the user’s natural language question and the domain-specific configuration required for query generation.

The system uses manual domain selection which lets the user to choose the domain that the question belongs to from a dropdown that is grouped under Wikidata and DBpedia section headers, as well as an automatic detection option.

Having a manual domain selection is important because the selected domain controls several technical components of the pipeline. First, it determines the SPARQL endpoint to which the generated query will be sent. For example, the diseases domain is connected to Wikidata, while the movies domain is connected to DBpedia. Therefore, selecting a domain does not only categorize the question conceptually, but also routes the query to the appropriate knowledge base.

Second, the domain selection affects the prefixes and vocabulary used in the generated SPARQL query. Wikidata and DBpedia use different schemas and different property structures. For instance, DBpedia commonly uses prefixes such as `dbo:`, while Wikidata relies on structures such as `wdt:` and `wd:`. If the wrong domain is selected, the model may generate a query with prefixes or properties that are not compatible with the selected endpoint.

Third, domain selection also influences the few-shot examples given to the LLM. These manually added extra example questions for each domain in order to help users understand what kinds of questions they can ask within the system. These questions are not generated automatically; instead, they are manually defined inside each domain configuration YAML file under the `example_questions` section.

For example, in the `wikidata_diseases.yaml` configuration file, I added questions such as:

- “List 10 diseases”
- “What are the symptoms of diabetes?”
- “Which drugs are used to treat malaria?”
- “Find diseases that affect the lungs”
- “Which diseases are caused by bacteria?”

For the Auto-detect option, the idea is that the LLM would read the user’s question, decide which domain it belonged to, and continue with that domain’s configuration.

The main motivation behind this feature was to reduce the risk of user error during domain selection. In the earlier version, the user had to manually choose the correct domain before asking a question. However, if the user selected the wrong domain, the system could send the query to an unsuitable endpoint or use an incompatible configuration.

With the new design, manual domain selection still remains available, but the user can also choose the Auto option. When Auto-detect is selected, the system first analyzes the user’s natural language question and asks the LLM to predict the most appropriate domain. After identifying the domain, the system automatically continues with the corresponding domain configuration, including the correct endpoint, prefixes, ontology hints, and few-shot examples.

For example, if the user asks:

Who directed Memento?

the system should recognize that this is a movie-related question and select the Movies (DBpedia) domain. After that, it uses the movie domain configuration to generate and execute the SPARQL query. The interface also informs the user about this decision by displaying a message such as:

Auto-detected domain: Movies (DBpedia)

This makes the system more transparent because the user can see which domain was selected automatically.

To test this feature, I used example questions from different categories, such as:

- “Who directed Memento?”
- “Top 10 planets”
- “Who influenced Kant?”

These examples were chosen because they clearly belong to different domains: movies, astronomy, and philosophy. This allowed me to check whether the automatic detection mechanism could distinguish between different types of user questions and route them to the correct domain configuration.

If the user selects a domain manually instead of using Auto-detect, the previous behavior remains unchanged. The selected domain’s description and example question buttons are still displayed in the sidebar, and the system directly uses that domain’s configuration. Therefore, the Auto-detect feature improves flexibility without removing the clarity and control provided by manual domain selection.

Robustness Under Wrong Domain Selection. I noticed that even when the wrong domain was selected, the system could still sometimes produce correct results. This can be considered an interesting strength of the system because it shows a certain level of robustness. For example, if the diseases domain is selected but the user asks a movie-related question, the query may still work if it is sent to Wikidata, since Wikidata also contains information about films. In such cases, the LLM can sometimes adapt to the user’s question and generate a reasonable query despite the imperfect configuration.

This behavior is likely related to the general knowledge and flexibility of the language model. Since models such as Llama 3.3 have broad knowledge about common knowledge bases like Wikidata, they may still produce meaningful SPARQL patterns even when the selected domain is not ideal. From a positive perspective, this suggests that the system is not completely dependent on a perfectly selected domain and can tolerate some user mistakes.

However, this observation also reveals a limitation. If the user asks a question that requires DBpedia-specific properties or structures, but the selected domain routes the query to Wikidata,

the generated query may fail or return incomplete results. Therefore, the system’s ability to work under a wrong domain should be seen as a useful robustness feature, but not as a replacement for correct domain selection.

4.2 Entity and Predicate Resolution

Early versions of the system embedded entity identifiers directly in the YAML domain configuration files: the few-shot examples and ontology-hint blocks contained hand-curated Wikidata QIDs and PIDs for the most common entities in each domain. This approach had a structural weakness. The hand-curated IDs covered only the entities the domain author had anticipated. Any question touching an entity outside that list caused the LLM to fall back on parametric memory, which, as documented in Section 4.3, frequently produces plausible but incorrect identifiers.

The improved design replaces the static YAML hints with a two-phase dynamic resolution step that runs at query time.

Phase 1 — LLM Extraction. Before the SPARQL generation prompt is assembled, a lightweight, low-temperature LLM call asks the model to read the user’s question and return the named entities and relational keywords that would need to be looked up in the knowledge graph. The output is parsed as a structured list of search terms rather than a query. For example:

```
Question : "Who directed Inception?"
Extracted: entities  = ["Inception"]
          relations = ["director"]
```

Listing 2: LLM extraction: input question and extracted terms.

Phase 2 — Wikidata API Lookup. Each extracted term is sent to the Wikidata `wbsearchentities` endpoint. The API returns a ranked list of candidates with their identifiers and labels; the system selects the top-ranked result for each term:

```
"Inception"  -> Q134773  (film, 2010)
"director"   -> P57      (director)

Injected hint block:
  KNOWN ENTITY: Inception = wd:Q134773
  KNOWN PROPERTY: director = wdt:P57
```

Listing 3: Wikidata API resolution and prompt injection.

Injection into the SPARQL prompt. The resolved identifiers are added as a grounded-hints block immediately before the user’s question in the generation prompt (see also Section 4.5). The LLM therefore receives verified, API-backed identifiers and is explicitly instructed not to substitute its own guesses. This occupies the same slot in the prompt that the old YAML-embedded IDs used, but is generated dynamically for each query rather than authored once at configuration time.

Entity lookups versus type constraints. The two-phase resolution step is most effective for questions about specific named entities: “Who directed Inception?”, “What is the capital of France?”, or “Which albums did Miles Davis record?”. In each case the LLM extracts a concrete proper noun, the Wikidata search API returns a single confident match, and the verified identifier is injected into the generation prompt. This is the failure mode that hurt the old approach most directly: if a named entity was not pre-listed in the YAML file the model had to guess its identifier, and guesses were often wrong. The old approach only produced correct results for such questions when the specific entity happened to have been hardcoded by the domain author.

Type-based list queries such as “List 10 galaxies” behave differently. Here the LLM extraction step identifies *galaxy* as a class concept rather than a named individual. The Wikidata API resolves it to the class identifier `wd:Q318`, which can then be injected as a type constraint (`?x wdt:P31 wd:Q318`) so the model does not have to recall the correct class ID from memory. This is still an improvement over the old method, which provided no useful hint at all for “List 10 galaxies” unless the domain author had manually added the galaxy class ID to the YAML. However, resolving the class does not change the fundamental query structure: the answer still comes from a broad `SELECT` over the endpoint, and that is where the timeout failures documented in Section 6.4 originate. In short, the two-phase resolution step makes the system more general and less dependent on manual curation, but it does not solve endpoint performance problems that arise from the scale of the knowledge graph itself.

The qualitative outcome of this change was therefore strongest for specific-entity questions, where the improvement replaced unreliable parametric recall with API-backed identifiers. For type-based list queries the gain is real but narrower: dynamic class resolution removes the need for hardcoded class IDs while leaving the broader query-execution challenges unchanged.

4.3 NL → SPARQL

The NL→SPARQL component is responsible for converting the user’s natural language question into an executable SPARQL query. This is the main query generation step of the system, because it allows users to ask questions in ordinary language without needing to know SPARQL syntax, RDF triples, prefixes, or ontology-specific properties.

The NL→SPARQL pipeline follows a prompt-driven structure. Entity and predicate resolution (Section 4.2) runs first and injects a block of verified Wikidata identifiers into the prompt. The system then builds the full generation prompt using the selected domain configuration, which provides the relevant endpoint, prefixes, ontology hints, and few-shot examples. The resolved entity block and the user’s question are appended last. This kind of prompt-based structure is well established in the prompting literature [3, 10], and our reason for adopting it was practical: in-context examples teach the model the schema vocabulary it should use, which is exactly what generic instruction-tuned models like Llama-3 or Qwen-2.5 cannot be expected to know without help [16].

After the LLM generates the query, the output is cleaned before execution. This cleaning step is necessary because instruction-tuned models often return SPARQL inside markdown formatting,

such as triple-backtick code fences. The system removes these extra formatting elements and normalises the prefix declarations so that the query can be executed correctly on the selected SPARQL endpoint.

If the cleaned query is valid but returns zero rows, the pipeline retries once by calling the LLM again with the same prompt at a higher temperature (0.7). Raising the temperature increases the randomness of the output, giving the model a chance to produce a structurally different query without any change to the instructions or examples. This is useful because zero-result queries are often caused by one wrong identifier rather than a completely wrong query structure, and a different generation at higher temperature can sometimes avoid the hallucinated term [8, 19]. The Model Arena tab uses a complementary strategy: there, the failed query is shown back to the model in a modified prompt that explicitly instructs it not to guess entity identifiers, which is a separate mechanism not part of the main pipeline.

In practice, this retry mechanism helps the system recover from cases where the first generated query was almost correct but used an incorrect or hallucinated identifier. Therefore, the NL→SPARQL pipeline combines LLM-based generation, domain-specific prompting, query cleaning, and a simple retry mechanism to produce more reliable executable SPARQL queries.

Entity-identifier hallucination. A common failure mode of LLM-generated SPARQL is *entity-identifier hallucination* [8]: the model produces queries that are syntactically valid but reference Q-IDs or P-IDs that do not actually exist or do not mean what the model thinks they mean. I ran into this very early when I asked the system “*What diseases affect our brain?*”. The model produced the pattern in Listing 4. The query is structurally correct, but Q9604 is not the Wikidata identifier for “brain”; the correct ID is Q1073. The query therefore executed without any error and returned zero rows, which made the bug invisible at first.

```
?disease wdt:P31 wd:Q12136 .
?disease wdt:P927 wd:Q9604 . # hallucinated entity
```

Listing 4: Hallucinated Q-ID: Q9604 is not the Wikidata identifier for brain (correct ID: Q1073).

To understand the difference, I also tested “*What are the symptoms of diabetes?*”, which worked reliably. The reason was that the Wikidata ID for diabetes, Q12206, was already in the few-shot examples, so the model could copy the ID instead of trying to recall it. The word “brain” was not in the examples, so the model had to guess, and it guessed wrong. This is not a moral failure of the model; it is exactly what the literature on parametric memory and hallucination predicts [8]. Large language models are not databases of exact entity identifiers, and Wikidata has millions of entities, so it is unrealistic to expect the model to remember all of them correctly.

To reduce this problem, we added an instruction to the prompt telling the model not to guess Q-IDs when it is not sure, and to use label-based matching instead. The same query is then rewritten as in Listing 5. This grounds the entity reference in the knowledge graph itself rather than in the model’s memory, which mirrors the design principle behind retrieval-augmented generation [?]: when the model is asked to draw on factual knowledge, the safer pattern is to

ground the answer in an external source rather than trust the model’s internal recall. It is a little slower than a direct Q-ID lookup, but it degrades gracefully on entities the model has not seen before, and it worked reliably for the rest of the questions in our test set. The dynamic entity resolution step described in Section 4.2 addresses this class of failure more directly: by resolving entity names via the Wikidata API before the generation prompt is assembled, the model receives a verified identifier and has no need to recall or guess it.

```
?organ    rdfs:label "brain"@en .
?disease wdt:P927   ?organ      .
```

Listing 5: Label-based fallback that avoids depending on memorised Q-IDs.

Fictional entities slipping through. A different version of the hallucination problem showed up later, when a query for “list 10 scientists” returned Spider-Man as one of the results. The query was syntactically valid and Wikidata does contain a Spider-Man entity (Q79037); the problem was that no type constraint restricted the answer to real human beings, so fictional characters with scientist-adjacent properties slipped through. The fix was to add the constraint `?person wdt:P31 wd:Q5` (instance of human) to the prompt template for person-related questions. The reason this works rather than just patching the example is that the constraint is added to the system prompt, so the model applies it to questions about people in general, not only to questions about scientists.

Duplicate results. Another early problem was duplicate rows. When I asked the system to list five films, it returned only two distinct rows, because DBpedia contains multiple versions of the same work, with different years or different language labels [9]. The same issue appears in Wikidata, where multi-valued properties produce Cartesian-product-style row duplication. I first added `SELECT DISTINCT` as a prompt instruction, but some models ignored it, so we eventually moved the fix into post-processing: `clean_sparql()` rewrites `SELECT` as `SELECT DISTINCT` on any query that has no aggregation function or `GROUP BY`. Aggregation queries are excluded because `SELECT DISTINCT COUNT(...)` is semantically wrong and would change the query’s intent. The reason for centralising this in post-processing rather than relying on the prompt is the same reason that informs much of the rest of our error handling: generative output needs a deterministic safety net, because no prompt instruction is followed 100% of the time.

4.4 SPARQL → NL

The SPARQL to Natural Language component is responsible for converting a generated or manually written SPARQL query into a clear human-readable explanation. This part of the system is important because SPARQL queries can be difficult to understand for users who are not familiar with RDF triples, prefixes, variables, ontology properties, or endpoint-specific vocabularies. By translating the query into natural language, the system helps the user understand what the query is asking, which entities and relationships are involved, and how the query will retrieve information from the selected knowledge base.

The SPARQL→NL direction started out with a purely rule-based translator. The translator

matched common triple patterns and dropped predicate labels into fixed templates. However, this approach produced explanations that were structurally correct but linguistically weak.

For example the query in Listing 6 produced the explanation in Listing 7.

```
SELECT DISTINCT ?director ?name WHERE {  
  dbr:The_Matrix dbo:director ?director .  
  ?director      rdfs:label   ?name      .  
  FILTER(LANG(?name) = "en")  
}
```

Listing 6: A test query about the director of *The Matrix* used to evaluate the rule-based verbaliser.

```
This query retrieves director and name where  
"The Matrix" "director of the film" director,  
and director "human-readable name (use with FILTER langMatches)"  
name, filtered by LANG(?name).
```

Listing 7: Rule-based output: structurally grounded but not readable.

This output shows the limitations of the rule-based approach. Although the system correctly detected that the query was related to the movie domain and successfully executed the query on DBpedia, the natural language explanation was not fluent. The sentence structure was broken, the entity `dbr:The_Matrix` was not converted into a clean natural language expression such as “The Matrix,” and the explanation of `rdfs:label` incorrectly included the full ontology hint instead of using it in a user-friendly way. The `FILTER` clause was also not explained clearly.

The main problem was that the rule-based translator worked with fixed templates. It tried to map SPARQL patterns into natural language using a simple “subject → predicate → object” structure. However, natural language does not always follow such a direct structure. The correct sentence form depends on the predicate and on which part of the triple is variable.

For example, a query pattern such as:

X dbo:director Y

should not always be translated literally as “X director Y.” If X is a specific movie and Y is a variable, the natural explanation should be:

This query asks for the director of X.

Similarly, a pattern such as:

X wdt:P737 Y

should be translated as:

X was influenced by Y.

or, if the object is fixed and the subject is variable:

This query asks for the entities that were influenced by Y.

This showed that a purely rule-based approach was not flexible enough for producing high-quality natural language explanations. Therefore, I decided to make the LLM the primary component for the SPARQL to Natural Language translation. The LLM is better suited for generating fluent, context-aware, and human-readable explanations because it can interpret the structure of the query and express it in natural English.

At the same time, I did not completely remove the rule-based component. Instead, I kept it as a secondary structural summary or fallback layer. This creates a hybrid approach: the LLM is used to generate the main natural language explanation, while the rule-based method can still provide a technical summary for validation or debugging purposes.

This improvement also makes the architecture more consistent. In the Natural Language to SPARQL direction, the system already uses the LLM as the main component, supported by prompt engineering and domain configurations. By using the LLM as the primary component for SPARQL to Natural Language as well, both directions of the system become more symmetrical: LLM-based translation is used for fluency and flexibility, while rule-based logic supports structure, verification, and transparency.

Overall, this change improves the user experience because the system no longer only describes the query mechanically. Instead, it can explain what the SPARQL query means in clear and natural English, while still preserving a technical layer for users who want to inspect the underlying query structure.

4.5 Prompt Engineering Strategies

The system uses four prompt engineering strategies, and they are all implemented inside the prompt builder and the YAML configuration files rather than in the pipeline logic. They were listed and explained in detail as follows:

- **Step-by-step reasoning.** The prompt asks the model to first step through the problem by considering the properties and entities to be used before composing the query. This is a less-structured version of chain-of-thought prompting [19] and works best with multi-clause questions. Furthermore, this is where the model must discover the appropriate join variables to commit to a structure.
- **Schema and ontology hints.** Each prompt contains the namespace prefixes and ontology hints block as part of the domain YAML. This means that the model will know which classes and properties are present in the target knowledge graph, such as that British authors should be queried using `dbo:nationality dbr:United_Kingdom`, instead of a free-text filter. The model hallucinates [8] names of plausible but non-existent properties without these hints.
- **Few-shot examples.** The six to seven curated question-SPARQL pairs in each domain YAML are of various difficulty. These are added to the beginning of each prompt and are the most successful method [3, 10]. They provide the model with the right prefix style, query structure, and property vocabulary for that particular domain.

- **Grounding of entities and predicates.** Before the generation prompt is assembled, a dedicated two-phase resolution step (Section 4.2) first uses the LLM to extract which entity names and relational keywords to search for, then resolves each term to a verified QID or PID via the Wikidata `wbsearchentities` API [17]. The confirmed identifiers are added as a grounded-hints block in the prompt, so the model receives API-backed IDs rather than having to recall them from parametric memory. This directly addresses the most common cause of syntactically valid queries that return no results [4].

4.6 Knowledge Graph

The sense of the Knowledge Graph component was aiming to demonstrate the SPARQL query results with the graph representation [7] instead of creating a new knowledge graph. While tabular results might suffice for examining individual relationships, we can use a knowledge graph representation to simplify the analysis of the response’s structure. This provides a more powerful visual representation. For example, a query that returns diseases and treatments, countries and capitals, or philosophers and their influences involves relationships between entities that would be better understood with the visual knowledge graph representation. The relations become explicit with the graph representation, the entities become nodes, and the bindings become edges [7]. In this way, the entities that are highly repeated or connected became more understandable.

The graph representation thus serves as an interpretability layer on top of the first result in the SPARQL result set. It is particularly helpful in an NL-to-SPARQL system [4, 6]: the user has not only to see the answer, but also to evaluate if the query generated is semantically correct. The application has an interactive visualisation and can also export the resulted graph for rendering as Turtle RDF [13]. This export allows the generated structure to be displayed with the application Protégé [11] or any other RDF tool, where the user can view the classes, individuals, object properties, the labels created by the application and its outputs.

Once the SPARQL query is completed, the application does not call the endpoint again to fetch any extra data to construct its graph. It rather makes use of just the bindings supplied by the query. This enables the graph representation to be directly traced to the query result, and prevents the visualisation from adding information which was not included in the answer. The graph is therefore not a standalone knowledge base, but rather a visualization of the result set.

The pipeline can be summarised as below:

1. The user provides a SPARQL query.
2. The application detects the appropriate endpoint.
3. The query is executed against Wikidata or DBpedia.
4. The returned rows are analysed to detect entity and label columns.
5. The system selects the graph mode: relational graph or star graph.
6. The result set is rendered as an interactive `pyvis` graph.

7. Optionally, the same graph structure is exported as Turtle RDF.

```
SPARQL query
-> endpoint detection
-> query execution
-> result rows
-> column detection
-> graph mode selection
-> PyVis visualisation
-> optional Turtle export
```

Listing 8: Pipeline for transforming SPARQL results into a graph representation.

One of the key steps in the workflow we use here is column detection. As mentioned earlier, SPARQL returns results in a tabular format. Therefore, the graph generator must clearly understand which columns represent entities and which represent labels. The application must thoroughly analyze the returned rows before creating nodes and edges. In Wikidata results, labels usually follow the conventions `?entityLabel`, for example `?diseaseLabel` or `?countryLabel`. Each of these columns is matched with the entity column, e.g., `?disease` or `?country`. On the other hand, with DBpedia results, labels frequently come back under the more general variable names of `?name` or `?title`. For this reason, the system leverages a value-based fallback: columns with values that look like URIs are considered entity columns, and textual columns are considered as labels.

This distinction is important because labels should identify nodes rather than becoming separate nodes themselves. If this is not done, the visualization may incorrectly display both the URI and the label as distinct entities. This can result in a graph that appears interconnected but does not accurately reflect the SPARQL result. By separating entity identifiers from human-readable labels, the system keeps the graph semantically closer to the original query output.

After the entity and label columns are specified the app chooses between two types of graph construction: relational graph mode, and star graph mode. This distinction is required as the structure of the result sets of SPARQL queries can differ. Some queries will include explicit relationships between two types of entities and others will return the flat list of entities. Either of these treatments would obscure or produce spurious relationships.

Relational graph mode. When there are at least two entity columns in the result set, the system will treat it as a relationship graph. The first entity column is considered to be the source node, and the second entity column is considered to be the target node. Then, each row returned forms a directed edge between the source and the target entity. This mode is suitable for queries whose answer is naturally a relation between entities, such as the relation between diseases and treatments, philosophers and influences, or countries and capitals.

```
?disease      -> ?drug
?philosopher  -> ?influence
?country      -> ?capital
```

For example, a disease-treatment query may return the following bindings:

```
Disease A -> Drug B
Disease A -> Drug C
Disease D -> Drug B
```

Here, the diseases and drugs are nodes, and the returned bindings are directed edges between them. Additionally, the relational mode eliminates duplicate source-target pairs, which helps prevent multiple duplicate edges. The size of the nodes is proportional to their degree, making high-degree nodes more visible. If the result set of the query includes an explicit edge label, this label is used in the graph, otherwise the renderer can extract the first direct Wikidata property identifier, e.g. `wdt:P2176` from the SPARQL query and use this as the edge label.

Star graph mode. If there is only one entity column in the result set, the system treats the query as a list query and not a relationship query. For instance, the following query just returns scientists and their labels:

```
?scientist    ?scientistLabel
```

In this instance, the returned scientists are not explicitly related to each other (entity to entity relationship). If they were directly connected to each other, they would therefore add a relation that is not present in the SPARQL result. The system, however, builds a synthetic central node, which takes the name of the variable, for example, **Scientists** and links each entity returned to the centre.

```
Scientists -> Albert Einstein
Scientists -> Marie Curie
Scientists -> Isaac Newton
```

This forms a star-shaped graph. This mode provides a visual structure to flat list results, but does not create false relationships between the entities listed.

The interactive graph visualisation is implemented with `pyvis`. PyVis generates an interactive HTML graph based on `vis.js`, which makes it suitable for embedding directly in the Streamlit interface. In the standalone Knowledge Graph tab, the user can choose between two layouts: a force-directed layout and a hierarchical layout.

```
Force-directed
Hierarchical
```

The force-directed layout is helpful for exploratory analysis since it distributes the nodes based on their connections and helps to identify clusters and highly connected nodes. The hierarchical structure is better suited when the query result is more directed (like source-to-target relationships). The visualisation also features dark mode, node search, auto zoom to fit, edge arrows and tooltips. The tooltips contain the human-readable label as well as the original URI or QID, so that the user can view the human-readable result and retain access to the original identifier. There are

also optional physics controls to control the spreading and attraction behaviour of the graph.

Relational graphs have an extra feature: Path Finder. Once a relational graph is displayed, the system constructs a directed graph based on the source and target columns currently displayed and using `networkx`. The user can then choose a starting node and an ending node.

```
From node  
To node
```

The system tries to find the shortest path between these two directed nodes. This path search is designed to only work for the current result set that is being rendered and does not re-query the entire Wikidata or DBpedia graph. That means the Path Finder displays connectivity in the result set that is currently rendered, not in the entire remote knowledge base. Once the path is discovered, the graph will be displayed again and the path will be shown in a different colour, which makes it easier to inspect visually.

The Knowledge Graph is not only available in the standalone tab. It is also integrated with the conversational interface. The user enters the following command and the system displays the last query results in memory:

```
/graph
```

The user can also provide a SPARQL query directly after the command:

```
/graph SELECT ...
```

In this case, the application first executes the supplied SPARQL query and then renders the returned rows as a graph. The same logic is reused in the agentic chat workflow through the `visualize_graph` tool. This tool retrieves a result artifact from memory, calls the same `build_pyvis_html` function, and stores the generated HTML as a `graph_<n>` artifact. Therefore, the Knowledge Graph tab, the `/graph` slash command, and the agentic graph tool all rely on the same graph construction logic instead of maintaining separate implementations.

After the graph is rendered, the user can export it as a Turtle file.

```
knowledge_graph.ttl
```

This export is implemented with `rdflib`. The goal is to convert the visualised result set into a small RDF/OWL-compatible graph that can be inspected outside the application. The export uses the following project namespace for synthetic classes, concepts, relations, and local fallback entities:

```
https://group-b-lamia.unige.ch/kg/
```

For relational graphs, the source variable and target variable are converted into OWL classes. The relation is inferred from the original SPARQL triple pattern when possible and is declared

as an `owl:ObjectProperty`. The export also assigns domain and range information to this property. Source and target nodes are represented as `owl:NamedIndividual` instances, and duplicate relation triples are removed. If label columns are available, the system writes them as `rdfs:label` values so that the exported graph remains readable in RDF tools.

For star graphs, the system uses a synthetic `Concept` class. It creates a central concept individual and an entity class based on the returned variable name. The central concept is connected to each returned entity through the relation `lamia:relation/has_member`. As in relational mode, readable labels are added with `rdfs:label` when they are available. This means that the exported Turtle file does not only contain raw identifiers; it also contains readable labels and a simple structure that can be inspected in Protégé or another RDF editor.

The benchmark executes twelve SPARQL queries covering both supported graph modes: eight star graph cases and four relational graph cases. It checks whether each query executes successfully, whether it returns results, whether the system correctly separates entity and label columns, and whether the detected graph mode matches the expected mode. It also validates the Turtle export by parsing it with `rdflib`, checking that `rdfs:label` values are readable text rather than raw URIs, verifying that star graph centre labels are correctly formed, and ensuring that relational graphs do not contain duplicate exported relation edges.

5 Implementation

5.1 Technology Stack

The prototype was implemented in Python and organised around a small set of libraries for the main components of the system. Streamlit was used to build the web interface, including the sidebar, domain selection, query input, generated SPARQL display, and result tables. The application also uses pandas in the interface layer to handle tabular outputs. Communication with the LLM services, entity lookup functions, and SPARQL endpoints is handled through HTTP requests using the requests library. The project dependencies also include RDF and graph-related libraries such as rdflib, networkx, and pyvis, which support the broader knowledge-graph-oriented structure of the prototype. This modular structure made it easier to connect the interface, the language model client, the SPARQL execution layer, and the evaluation scripts.

Component	Role in the system	Library
UI	Web interface, sidebar, tabs, result display	Streamlit
Tabular results	Display and handling of result tables	pandas
LLM client	Calls to Hactar and Hugging Face APIs	requests
Entity lookup	External lookup/API calls	requests
SPARQL execution	Endpoint queries and JSON responses	requests
Configuration	Domain YAML files and settings	PyYAML
RDF / graph tooling	RDF and graph-related support	rdflib, networkx, pyvis

Table 1: Summary of the libraries used in the prototype.

5.2 Hactar LLM Service

Hactar is the local LLM service hosted by the University of Geneva. It exposes an OpenAI-compatible interface to several locally deployed instruction-tuned models, including `llama3.3:latest` [16], `qwen3:latest`, and `mistral:latest`. Two things make Hactar a good fit for a KOS course project. Firstly all prompts stay inside the university infrastructure, which matters for privacy, and secondly the served models are stable releases, which makes prompt-engineering experiments reproducible across the lifetime of the project. The downside is throughput. Hactar serves a shared user base and was sometimes unavailable during the evaluation phase, which is why we later added HuggingFace Inference Endpoints as a fallback backend.

I first made sure that the system was working properly, and then I tested the available models to get an idea. I evaluated several available large language models within the Hactar environment in order to compare their response quality, reasoning capabilities, and suitability for SPARQL query generation and natural language understanding tasks. After that, I created an API key from Hactar’s settings page and an access key on GitLab so that I could push code from the terminal. I wrote the first test script in VS Code, ran it in the integrated terminal, and pushed it to GitLab.

The first test code had the goal to verify that I could establish a stable connection to Hactar, find the right endpoint, and confirm that the LLM could produce SPARQL that runs against Wikidata. I tried a few different endpoint paths and settled on `/api/chat/completions`. The initial tests showed that the model was able to generate meaningful SPARQL queries, which gave us the green light to build the rest of the system around this backend. The `llm_client` module wraps this endpoint and accepts the model name as a parameter, so the same client code transparently serves both Hactar and HuggingFace.

5.3 HuggingFace

Due to repeated instability with Hactar during development and evaluation, we needed an alternative way to access large language models. For this reason, we used Hugging Face, an online platform that provides access to many open-source and instruction-tuned language models. Hugging Face was a practical choice because it allowed us to test models such as Qwen and Llama through a free and accessible interface, without depending entirely on the availability of the Hactar service. This made it possible to continue the experiments and compare model outputs even when Hactar was not responding reliably.

HuggingFace Inference Endpoints were accessed via HTTP using the same `requests`-based `llm_client` module that calls Hactar, so no additional dependencies were required. The two models used for evaluation were `Qwen/Qwen2.5-Coder-32B-Instruct` and `meta-llama/Meta-Llama-3-8B-Instruct`, both served through the HuggingFace Inference API.

5.4 Handling Endpoint Quirks

Several practical issues showed up during development. Each was fixed at the level where it happened (prompt, post-processing, or domain config) rather than by adding more LLM machinery. There is a common thread running through all of them: generative output needs deterministic

post-processing, because no prompt instruction is followed 100% of the time. The cases below are the ones that shaped this principle.

Markdown fence residue. Instruction-tuned models like to wrap their output in triple-backtick fences. The `sparql_executor.clean_sparql` routine handled most of these, but some edge cases (a trailing backtick after `LIMIT 10`) slipped through and made Virtuoso return SP030 syntax errors. The fix was a simple defensive step: `text=text.replace("`", "").strip()`. This removes the class of failures, at the cost of also stripping any deliberate backticks (which never occur in valid SPARQL anyway).

Slow label resolution on Wikidata. On Wikidata, label resolution through `rdfs:label` combined with `FILTER(LANG(?name) = "en")` is functionally correct but very slow, because the engine has to scan every literal in the graph [17]. The same query phrased with the dedicated label service (`SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }`) usually finishes in a fraction of the time, because the service is optimised for this exact pattern. I promoted the label-service pattern in the few-shot examples so that the model is biased toward the fast path. This one change removed several timeout failures from the evaluation set (Section 6.4).

Class granularity in the DBpedia books domain. The example question “*List 10 novels*” exposed a subtler problem. The natural starting point, `?book a dbo:Novel`, returns a strange mix of novella articles, concept pages, and DBpedia misclassifications. The query is technically working but the results are obscure and not very book-like. Tightening the pattern to require both `dbo:Novel` and `dbo:author` reduces the noise but produces too few results, because the `dbc:Novels` category in DBpedia is almost empty; most individual novels are filed under more specific sub-categories such as `dbc:English_novels` or `dbc:American_novels` [9]. I tried using the broader `dbo:Book` together with the `dbo:author` filter, but this still cut too many results. In the end I went with `?book a dbo:WrittenWork` as a compromise: it is technically broader than “novel” (it includes short stories and other written works) but it is densely populated in DBpedia and, together with `ORDER BY RAND()` and `DISTINCT`, produces a reasonable sample for the example query. The lesson is that prompt engineering alone cannot fix data-coverage problems in the target graph: sometimes the only honest fix is to weaken the query.

Sports domain entity-ID mismatch. Queries such as “*List 10 tennis players*” consistently returned basketball players, and “*List 10 soccer players*” returned nothing. The generated SPARQL was syntactically valid but semantically wrong, because the YAML configuration had mapped sport names to the wrong Wikidata entities. After auditing `configs/wikidata_sports.yaml` and correcting the identifiers (`wd:Q847` for tennis, `wd:Q5372` for basketball, `wd:Q2736` for association football, and `wd:Q937857` for the “association football player” occupation) the queries returned the expected results. This case makes the same point but more strongly: *the LLM faithfully generates what the configuration tells it to generate*. A wrong identifier in a YAML file becomes a wrong predicate in the query and a wrong answer to the user. From an engineering point of view this means that the configuration files have to be treated as production code, not as documentation [3, 10].

Repeated queries returning the same rows. When the same natural-language question (e.g. “*list 10 thriller series*”) was asked multiple times, the system returned identical rows in the same order every time. There were two compounding reasons. First, the LLM was called at low temperature and produced the same SPARQL string every time. Second, DBpedia’s public Virtuoso endpoint caches results by exact query string, so even adding `ORDER BY RAND()` to the query had no effect on cached responses. We fixed this with two changes inside `sparql_executor.py`. `clean_sparql()` now automatically inserts `ORDER BY RAND()` before any `LIMIT` clause when the query has no existing ordering and no aggregation, so that fresh executions return a random sample. And `execute()` prepends a unique random-integer comment (`#1482736591`) to every request, which leaves the SPARQL semantics unchanged but makes the query string unique on each call so that the endpoint’s result cache is bypassed. Combining the two changes makes the system both feel and behave non-deterministic to the user, which is what they expect from a question like “list 10 random novels”.

6 Evaluation

6.1 Evaluation Setup

For a long period of the evaluation phase, Hactar was unreachable, so I had to start running the experiments reported here against HuggingFace Inference Endpoints, which is the fallback backend we integrated for exactly this situation. Two models were used:

`Qwen/Qwen2.5-Coder-32B-Instruct`, a code-tuned model with 32 billion parameters, and `meta-llama/Meta-Llama-3.1-70B-Instruct`, a smaller general-purpose instruction-tuned model [16]. Both models were run through the same domain configurations, the same prompt templates, and the same post-processing pipeline, so any difference in their results reflects the model itself rather than the surrounding code.

The evaluation is guided by the following research questions:

- To what extent can local LLMs generate executable SPARQL queries for typical KOS-style questions?
- How do the two candidate models perform on a multi-domain benchmark?
- When failures occur, are they mainly caused by model errors, prompt design limitations, or endpoint infrastructure issues?

Each model was first tested with a smaller quick-test pass during prompt iteration, and then with a full 16-question benchmark covering the four configured domains (books, biomedical, astronomy, philosophy). Four metrics are reported per query. Execution Accuracy (EA) is a binary flag for whether the generated SPARQL parses and executes against the endpoint within the timeout. Result Accuracy (RA) is a binary flag for whether the query returns a non-empty result set; RA implies EA but not the other way around. Component F1 is a token-level F1 score over the semantic components of the query (entities, predicates, filters) compared against a gold reference. BLEU is also reported for comparability with earlier work on neural SPARQL generation [20].

The evaluation harness lives in `tests/evaluate.py` and writes per-question outcomes to JSON so that failures can be inspected one by one.

6.2 Question Set

The 16-question benchmark is balanced across the four domains and across query types. Each question is tagged with a difficulty label that mirrors the six-way classification used in the UI (*Simple, Lookup, Filter, Join, Aggregate, Path*). The distribution is summarised in Table 2.

Domain	Endpoint	# Questions	Tags covered
Books	DBpedia	4	Simple, Lookup, Filter
Biomedical	Wikidata	4	Lookup, Filter, Join
Astronomy	Wikidata	4	Simple, Aggregate, Path
Philosophy	Wikidata	4	Lookup, Join, Path

Table 2: Distribution of the 16 benchmark questions across domains and difficulty tags.

6.3 Quantitative Results

Model	EA	RA	Comp. F1	Failures
Qwen2.5-Coder-32B-Instruct	13/16 (81.2%)	13/16 (81.2%)	0.979	3
Meta-Llama-3-8B-Instruct	13/16 (81.25%)	12/16 (75.0%)	0.979	4

Table 3: Aggregate performance on the 16-question benchmark. EA = Execution Accuracy; RA = Result Accuracy; Comp. F1 = average Component F1 across all 16 queries.

6.4 Qualitative Analysis

The failure cases turned out to be more informative than the aggregate numbers. Table 4 lists all seven failures across the two models, together with what I think the most likely root cause is.

Model	ID	Question	EA	RA	F1	Likely root cause
Qwen	A02	List 10 galaxies	✗	✗	1.00	Wikidata (rdfs:label) timeout
Qwen	P01	List 10 philosophers	✗	✗	1.00	Wikidata (rdfs:label) timeout
Qwen	B01	List 10 novels	✗	✗	–	Markdown fence residue (since fixed)
Llama	A02	List 10 galaxies	✗	✗	1.00	Wikidata timeout
Llama	P01	List 10 philosophers	✗	✗	1.00	Wikidata timeout
Llama	B02	List books by Orwell	✗	✗	1.00	DBpedia timeout (BLEU = 1.00)
Llama	M03	Who directed <i>Inception</i> ?	✓	✗	1.00	DBpedia: 0 results despite valid query

Table 4: All seven failures from the 16-question benchmark. Component F1 is the per-query F1 reported by the harness; “–” indicates that the query failed to parse and so could not be scored.

For **Qwen**, none of the three failures is really the model producing wrong SPARQL. Two of them (A02 and P01) are Wikidata timeouts caused by using `rdfs:label` with a language filter, as I described earlier. The generated SPARQL is semantically correct (Component F1 = 1.00 in both cases) but the query plan does not finish within the public 30-second limit. The third failure (B01, “List 10 novels”) is the markdown residue bug. The model appended a trailing triple-backtick fence to its output, the cleaner missed it, and Virtuoso returned an SP030 syntax error. This is the only *real* bug among the three failures, and it was fixed during the evaluation iteration. Component F1 of 0.979 across the run is a good sign that the model is producing semantically correct queries; the failures are almost entirely infrastructure problems.

For **Llama-3-8B**, three of the four failures are also timeouts (A02, P01, B02). The B02 case is the most striking one: the generated query has BLEU = 1.00 against the gold reference, which means it is essentially identical to what we would have written by hand, and yet the DBpedia endpoint still returned a timeout. So this failure is almost certainly an endpoint-load issue rather than a model issue. The remaining failure (M03, “Who directed *Inception*?”) is the only *genuine* semantic failure I observed across all 32 question-model pairs in the benchmark. The query parses and executes successfully (EA = ✓) but returns zero rows (RA = ✗), despite a perfect Component F1. The likely reason is that the model went for a property/entity pattern that does not quite match how DBpedia actually stores the director relation for that resource. So the model headed in the right direction semantically, but produced a pattern that does not align with the underlying data layout. This is the most interesting failure of the whole run and the one most worth following up on.

Putting these together, the headline gap between Qwen (RA 81.2%) and Llama (RA 75.0%) is mostly driven by infrastructure-level failures that both models share. If the four Wikidata- and DBpedia-induced timeouts are taken out, Qwen has one remaining failure (the markdown bug, now fixed) and Llama has one remaining failure (M03), which makes the gap much smaller than the table suggests. For this reason I treat the 16-question set as a *smoke test* rather than a

definitive ranking.

6.5 LLM as a Judge

The four metrics used in Section 6.3, Execution Accuracy, Result Accuracy, Component F1, and BLEU, are all *structural*. They tell us whether the query runs, whether rows come back, and whether the right tokens appear in the generated text. What none of them tell us is whether the rows that do come back actually answer the user’s question. A query that returns ten rows of random diseases still gets $EA = \checkmark$, $RA = \checkmark$, and a high Component F1, even if its answers have nothing to do with what was asked. This is the gap that motivated the LLM-as-a-Judge layer [22] implemented in `evaluate_v2.py`.

Judge design. The judge is a separate model, not the one being evaluated, to avoid the self-serving bias that comes from asking a model to rate its own outputs [22]. It receives four things: the original question, the generated SPARQL query, a sample of up to ten result rows in JSON, and the gold reference query for context. From these it scores the answer on three independent 0–5 axes:

- **Relevance:** do the returned items belong to the type the user asked for?
- **Correctness:** are the specific values factually right, with no wrong entities or nonsense results?
- **Completeness:** does the result cover the scope implied by the question?

After scoring each axis it gives a single verdict: `CORRECT`, `PARTIAL`, or `INCORRECT`. The three axis scores are averaged and normalised to $[0, 1]$ to give a scalar `judge_score` that can be combined with the other metrics.

Row-level judge. Beyond the overall verdict, a second pass looks at each returned row individually. The row-level judge is given the question, a sample of the gold result rows as a reference, and the generated rows numbered in order. For each one it returns `CORRECT` or `INCORRECT` with a short reason. The fraction of correct rows gives a `row_precision` score. This is useful because aggregate metrics can easily hide problems. A query that returns five correct rows and five rows featuring a fictional character still scores $RA = \checkmark$ and $\text{Component F1} = 1.00$, but `row_precision` = 0.50 makes the problem visible right away. This is the same kind of noise that showed up in case A03 (Section 6.4), where 97 rows came back for “*Which moons orbit Jupiter?*” with no type constraint to filter out non-moon objects.

Composite Quality Score. The judge score feeds into a Composite Quality Score (CQS), which is a weighted average of all available metrics. Table 5 shows the two sets of weights: one for when the judge is available and a fallback for when it is not. Result Set Match (RSM-F1), which compares the actual result sets of the generated and gold queries, and the judge score together account for 0.55 of the total weight. The reasoning is that knowing whether the right answers came back matters more than checking whether the query tokens look similar to a reference.

When a metric cannot be computed, for example if the gold reference query also times out making RSM impossible to score fairly, its weight is redistributed proportionally across the rest so that CQS stays in $[0, 1]$.

Metric	With judge	No judge
Execution Accuracy (EA)	0.10	0.10
Result Accuracy (RA)	0.10	0.10
Component F1	0.10	0.15
Result Set Match F1 (RSM)	0.30	0.45
Ground Truth Coverage (GTC)	0.15	0.20
LLM Judge score	0.25	n/a

Table 5: CQS weight sets in `evaluate_v2.py`. Weights of skipped metrics are redistributed proportionally at runtime so CQS always sums to one.

Relation to observed failures. Going back to the failures in Section 6.4, the judge layer adds detail that the structural metrics cannot provide. For case M03 (“*Who directed Inception?*” with Llama, EA = ✓, RA = ✗), the result set was empty despite the query executing. The judge would give correctness = 0, completeness = 0, and a verdict of INCORRECT, and the reasoning field would point to the likely cause: the generated pattern did not match how DBpedia actually stores the director relation for that resource. For the timeout cases (A02, P01, B02) the judge is simply not called because EA is already false, but the retry log in `evaluate_v2.py` records whether the failure was transient, which keeps the distinction between an endpoint issue and a real model error visible in the results. Case A03 (“*Which moons orbit Jupiter?*”) is probably the most interesting one for the judge to handle: it returned 97 rows and passed all structural checks with Component F1 = 0.67, yet the expected Wikidata class `wd:Q25257` (natural satellite) was missing from the query. A row-level pass would flag individual rows that are not actually moons, giving a concrete number for the noise that a single F1 score can only hint at.

7 Discussion

7.1 Limitations

This project has several limitations. First, the system depends heavily on the quality and consistency of the selected knowledge graph. DBpedia and Wikidata are large and useful, but their schemas are not always complete or uniform, so a correct looking query can still return incomplete, noisy, or unexpected results. Second, the models are not fine-tuned for this task; they are guided through domain specific prompts and examples, which means their outputs can still vary or fail on ambiguous questions. Third, the dynamic entity resolution step (Section 4.2) reduces identifier hallucination but does not eliminate it: the Wikidata search API ranks candidates by textual similarity and can return an incorrect top result for ambiguous names—a film title that also matches a book, or a person’s name shared by multiple individuals. Finally, public SPARQL endpoints can be slow or temporarily unavailable, so some failures are caused by timeout or

endpoint instability rather than by the generated query itself.

7.2 Future Work

Several directions could extend this work. The Model Arena could be connected to a broader benchmarking pipeline that automatically collects ratings across a fixed question set and produces aggregate statistics, rather than relying on manual per-session feedback. Web-based search grounding, where an LLM retrieves live entity identifiers before constructing the query, could reduce hallucinated Q-IDs and improve precision on questions about recent or niche topics. On the evaluation side, a larger annotated test set covering more domains and question types would give a more reliable picture of where each model succeeds and where it fails. Finally, fine-tuning a small model on the curated few-shot examples and query–result pairs collected through the arena could yield a system that is both faster and more consistent than prompting general-purpose LLMs.

8 Conclusion

This project set out to make two large knowledge graphs, Wikidata and DBpedia, reachable without writing SPARQL by hand, and to do so in both directions: turning a natural-language question into an executable query, and turning a query back into a readable explanation. We built a working prototype around local language models, a per-domain configuration system, a hybrid rule-and-LLM verbaliser, an interactive knowledge-graph view with Turtle export, and a multi-model arena for comparing models on the same question.

The evaluation showed that both Qwen2.5-Coder-32B and Llama-3-8B can generate executable and semantically correct SPARQL for a majority of questions across four domains, with most failures attributable to endpoint timeouts or data-coverage gaps in the knowledge graph rather than model errors. The two-phase entity resolution pipeline, domain-specific few-shot prompting, and deterministic post-processing together proved more reliable than any single component alone. From a knowledge organisation perspective, this confirms that LLM-based query generation can meaningfully lower the access barrier to structured knowledge, but that robust deployment still requires careful domain configuration, transparent evaluation, and deterministic safety nets on the output side. The Model Arena and LLM-as-a-Judge layer provide a reusable framework for ongoing model comparison as new models become available on Hactar or through the HuggingFace backend.

References

- [1] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. DBpedia: A nucleus for a web of open data. In *Proc. 6th International Semantic Web Conference (ISWC)*, pages 722–735, 2007. doi: 10.1007/978-3-540-76298-0_52.
- [2] Christian Bizer, Tom Heath, and Tim Berners-Lee. Linked data — the story so far. *In-*

- ternational Journal on Semantic Web and Information Systems*, 5(3):1–22, 2009. doi: 10.4018/jswis.2009081901.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
 - [4] Dennis Diefenbach, Vanessa Lopez, Kamal Singh, and Pierre Maret. Core techniques of question answering systems over knowledge bases: A survey. *Knowledge and Information Systems*, 55(3):529–569, 2018. doi: 10.1007/s10115-017-1100-y.
 - [5] Gail Hodge. Systems of knowledge organization for digital libraries: Beyond traditional authority files. Council on Library and Information Resources, 2000. <https://www.clir.org/pubs/reports/pub91/>.
 - [6] Konrad Höffner, Sebastian Walter, Edgard Marx, Ricardo Usbeck, Jens Lehmann, and Axel-Cyrille Ngonga Ngomo. Survey on challenges of question answering in the semantic web. *Semantic Web*, 8(6):895–920, 2017. doi: 10.3233/SW-160247.
 - [7] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021. doi: 10.1145/3447772.
 - [8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.
 - [9] Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N. Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick van Kleef, Sören Auer, and Christian Bizer. DBpedia – a large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web*, 6(2):167–195, 2015. doi: 10.3233/SW-140134.
 - [10] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023. doi: 10.1145/3560815.
 - [11] Mark A. Musen. The Protégé project: A look back and a look forward. *AI Matters*, 1(4): 4–12, 2015. doi: 10.1145/2757001.2757003.
 - [12] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems*, 34(3):16:1–16:45, 2009. doi: 10.1145/1567274.1567278.
 - [13] RDFLib Team. RDFLib: A Python library for working with RDF, 2024. <https://rdflib.readthedocs.io>.

- [14] Streamlit Inc. Streamlit: A faster way to build and share data apps, 2024. <https://streamlit.io/>.
- [15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [16] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [17] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi: 10.1145/2629489.
- [18] W3C SPARQL Working Group. SPARQL 1.1 query language. W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-query/>.
- [19] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [20] Xiaoyu Yin, Dagmar Gromann, and Sebastian Rudolph. Neural machine translating from natural language to SPARQL. In *Future Generation Computer Systems*, volume 117, pages 510–519, 2021. doi: 10.1016/j.future.2020.12.013.
- [21] Marcia Lei Zeng. Knowledge organization systems (KOS). In *Knowledge Organization*, volume 35, pages 160–182. Ergon Verlag, 2008.
- [22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

A Repository and Reproducibility

The source code of the project is available in the [GitLab repository](#).

To reproduce the project locally, the repository can be cloned and the required Python dependencies can be installed with:

```
git clone https://gitlab.unige.ch/kos_2026/group_b_lamia.git
cd group_b_lamia
pip install -r requirements.txt
```

The application requires environment variables for the LLM provider, especially when using Hactar. These can be stored in a local `.env` file, following the structure of the provided `.env.example` file. After the environment is configured, the Streamlit application can be launched with:

```
streamlit run app.py
```